Unit-8
Web application using Django framework

Mr. Nilesh Parmar

Assistant Professor, Dept. of Computer Engg.

UVPCE, Ganpat University, Mehsana

Django framework

- Django is a fully featured web framework written in Python.
- A web framework is a software which helps you to build flexible, scalable, and maintainable dynamic website, web app, and web services.
- It provides a set of tools and functionalities that solves many common problems associated with Web development, such as security features, database access, sessions, template processing, URL routing, internationalization, localization, and much more.
- It is a high-level web framework which allows performing rapid development. The primary goal of this web framework is to create complex database-driven websites.
- Different web frameworks like Django are Zend for PHP, Ruby on Rails for Ruby, etc.

Why to use Django?

- Some of the biggest web sites uses Django are: Instagram, Mozilla, Spotify, National Geographic, Pinterest, Open Stack, Dropbox, Bitbucket, NASA.
- **Advantages of Django:**
- *Object-Relational Mapping (ORM) Support* − Django provides a bridge between the data model and the database engine, and supports a large set of database systems including MySQL, Oracle, Postgres, etc. Django also supports NoSQL database through Django-nonrel fork. For now, the only NoSQL databases supported are MongoDB and google app engine.
- *Multilingual Support*− Django supports multilingual websites through its built-in internationalization system. So you can develop your website, which would support multiple languages.
- *Framework Support*− Django has built-in support for Ajax, RSS, Caching and various other frameworks.
- *Administration GUI*− Django provides a nice ready-to-use user interface for administrative activities.
- *Development Environment*− Django comes with a lightweight web server to facilitate end-to-end application development and testing.

Features of Django

- **Rapid Development:** Django was designed with the intention to make a framework which takes less time to build web application. The project implementation phase is a very time taken but Django creates it rapidly.
- **Secure:** Django takes security seriously and helps developers to avoid many common security mistakes, such as SQL injection, cross-site scripting, cross-site request forgery etc. Its user authentication system provides a secure way to manage user accounts and passwords.
- **Scalable:** Django is scalable in nature and has ability to quickly and flexibly switch from small to large scale application project.
- **Fully loaded:** Django includes various helping task modules and libraries which can be used to handle common Web development tasks. Django takes care of user authentication, content administration, site maps, RSS feeds etc.

Cntd…

- **Versatile:** Django is versatile in nature which allows it to build applications for different-different domains. Now a days, Companies are using Django to build various types of applications like: content management systems, social networks sites or scientific computing platforms etc.
- **Open Source:** Django is an open source web application framework. It is publicly available without cost. It can be downloaded with source code from the public repository. Open source reduces the total cost of the application development.
- **Vast and Supported Community:** Django is an one of the most popular web framework. It has widely supportive community and channels to share and connect.

History of Django

- 2003 − Started by Adrian Holovaty and Simon Willison as an internal project at the Lawrence Journal-World newspaper.
- 2005 − Released in July 2005 and named it Django, after the jazz guitarist Django Reinhardt. Version 0.9 was released in November, 2005.
- 2008- Version 1.0 was released. Django Software Foundation (DSF) started maintaining Django.
- 2013- Python 3 Support provided, configurable user model
- 2017 − First Python 3-only release, Simplified URL routing syntax, Mobile friendly admin.
- Current − Django is now an open source project with contributors across the world. Its current stable version is 2.0.3 which was released on 6 March, 2018.

Django – Design Philosophies

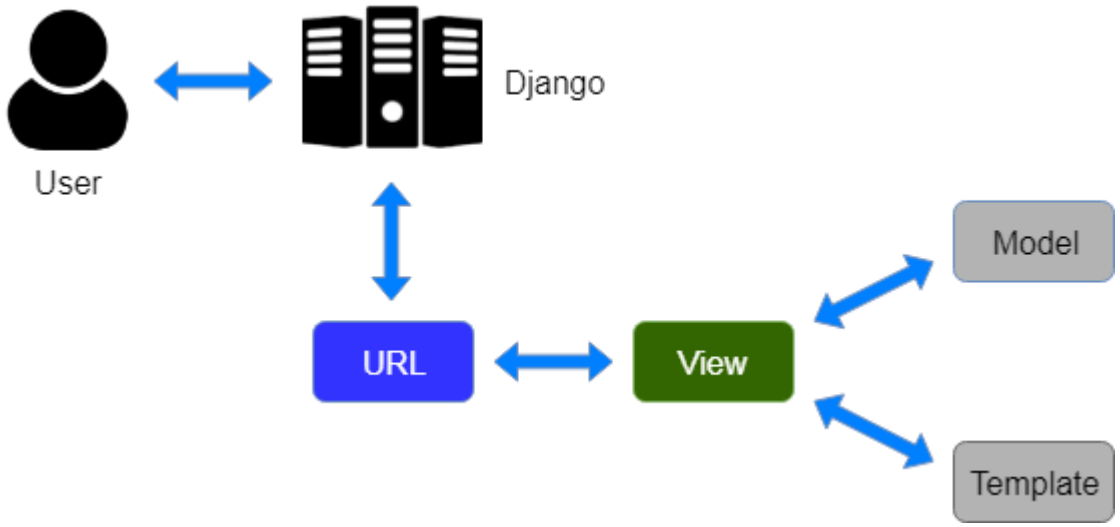
- Django comes with the following design philosophies −
- Loosely Coupled − Django aims to make each element of its stack independent of the others.
- Less Coding − Less code so in turn a quick development.
- Don't Repeat Yourself (DRY) − Everything should be developed only in exactly one place instead of repeating it again and again.
- Fast Development − Django's philosophy is to do all it can to facilitate hyper-fast development.
- Clean Design − Django strictly maintains a clean design throughout its own code and makes it easy to follow best web-development practices.

MVT Architecture

- Django follows MVT architecture (software design pattern).
- To understand MVT architecture, let’s first talk about widely known MVC architecture.
- MVC stands for *Model View Controller*. Model, view and controller are 3 components used for developing the web applications.
- **Model**– Model is used for storing and maintaining your data. It is the backend where your database is defined.
- **Views**–In Django templates, views are in html. View is all about the presentation and it is not at all aware of the backend. Whatever the user is seeing, it is referred to a view.
- **Controller**– Controller is a business logic which interacts with the model and the view.

Cntd..

- MVT stands for Model-View-Template.
- The main difference between MVC and MVT is that, in MVT there is no separate controller. Django itself manages the Controller part (software code that controls the interactions between the Model, View and template).
- The template is a HTML file mixed with Django Template Language (DTL).



Django MVT Architecture

Cntd..

- **Explanation of Diagram:** A user requests for a resource to Django. Django acts as a controller and checks to the available resource in URL.
- If URL maps, a view is called which interacts with model and template.
- Django then sends the response to user.
- View is your front end which interacts with template and the model will be used as a backend.
- Let’s take an example, you want to write several static html forms which prints hello user 1, hello user2 and so on. With template, you will be having only one file that prints ‘hello’ along with the variable name. Now this variable will be substituted in that particular template with different values of user1, user2 and so on. That’s the magic of template, you don’t need to rewrite the code again and again!

Components of Django

- **Form:** Django has a powerful form library which handles rendering forms as HTML. The library helps in validating submitted data and converting it to Python types.
- **Authentication:** It handles user accounts, groups, cookie-based user sessions, etc.
- **Admin:** It reads metadata in your models to provide a robust interface which can be used to manage content on your site.
- **Internationalization:** Django provides support for translating text into various languages, locale-specific formatting of dates, times, numbers, and time zones.
- **Security:** Django provides safeguard against the following attacks:
  - Cross-Site Request Forgery (CSRF)
  - Cross-site scripting
  - SQL injection
  - Clickjacking
  - Remote code execution

Two important concepts of Django

- **app:** It is similar to a web page that does something. An app usually is composed of a set of models (database tables), views, templates, tests. App is like a webpage of the website.
- **project:** Its is a collection of configurations and apps. One project can be composed of multiple apps, or a single app. Project can be considered as entire website.

Installation

- The basic setup consists of installing **Python** and **Django**
- We will use Visual studio code for simplicity in development of web application. Download and install visual studio code.
- **Note:** Virtual environment can also be created for coding.
- Install Django framework using following command in vs code Terminal:

  pip install django

  (Keep internet working)
- Install extensions in vs code- Django and python.

Creation of a Django Project

- Use following command in vs code terminal:

django-admin startproject project_name

- Go inside project directory by writing:

cd project_name

- Now, we need to run Django project for first time:

python manage.py runserver

(when you run project first time, also execute: python manage.py migrate)

- Create an app (webpage) in Django:

python manage.py startapp app_name

Files of Django Project

- manage.py: a shortcut to use the django-admin command-line utility. It’s used to run management commands related to our project. We will use it to run the development server, run tests, create migrations etc.
- project_name/ – It is the actual Python package for your project. It is used to import anything, for example – project_name.urls.
- __init__.py Tells Python to treat the current directory as a Python package.
- settings.py: This file contains all the configuration of Django project. We can manage and change project settings through this file.
- urls.py: this file is responsible for mapping the routes and paths in our project. For example, if you want to show something in the URL /about/, you have to map it here first.
- wsgi.py: WSGI stands for "Web Server Gateway Interface". This file is a simple gateway interface used for web application deployment. We will not bother about it.
- asgi.py- It stands for "Asynchronous Server Gateway Interface". It is a successor of wsgi.py.

Files of an App of Django Project

- admin.py contains settings for the Django admin pages.
- apps.py contains settings for the application configuration.
- models.py contains a series of classes that Django’s ORM converts to database tables.
- tests.py contains test classes.
- views.py contains functions and classes that handle what data is displayed in the HTML templates.
- Other files works in similar way to that of Project files.

Creating an admin user

- To Create an admin user, follow steps given below:

1) Type this command in vscode Terminal- python manage.py migrate

2) Then type in vscode Terminal- python manage.py createsuperuser

3) Enter desired username and press enter

4) You will then be prompted for your desired email address.

5) Then you will be asked to enter your password twice, the second time as a confirmation of the first.

6) On creation of user, it will display “Superuser created successfully.”

To understand, website/web application development using Django framework, refer my video: https://youtu.be/dvU4QY08D24